W2D5

Created time	October 21, 2026 5:25 PM

Q1.

```
CREATE DATABASE IF NOT EXISTS britmart_performance;

USE britmart_performance;

CREATE TABLE orders_perf (
order_id INT PRIMARY KEY,
customer_id INT NOT NULL,
order_date DATE NOT NULL,
city VARCHAR(50) NOT NULL,
amount_gbp DECIMAL(10,2) NOT NULL,
status VARCHAR(20) NOT NULL
);

INSERT INTO orders_perf VALUES
(1, 101, '2026-08-01', 'London', 45.50, 'complete'),
(2, 102, '2026-08-01', 'Manchester', 18.00, 'cancelled'),
(3, 101, '2026-08-02', 'London', 62.25, 'complete'),
(4, 103, '2026-08-02', 'Leeds', 30.00, 'complete'),
(5, 104, '2026-08-03', 'Bristol', 55.50, 'complete'),
(6, 102, '2026-08-03', 'Manchester', 12.00, 'pending'),
(7, 101, '2026-08-04', 'London', 80.00, 'complete'),
(8, 105, '2026-08-04', 'Leeds', 20.00, 'complete'),
(9, 106, '2026-08-05', 'Bristol', 100.00, 'complete'),
(10, 102, '2026-08-05', 'Manchester', 40.00, 'complete');
```

Q2.

```
EXPLAIN
SELECT
order_id,
order_date,
amount_gbp
```

```
FROM orders_perf
WHERE customer_id = 101;
```

Q3.

```
CREATE INDEX idx_orders_customer
ON orders_perf(customer_id);

EXPLAIN
SELECT
order_id,
order_date,
amount_gbp
FROM orders_perf
WHERE customer_id = 101;
```

Q4.

When is an index most likely to help?

B. A query filters a large table using a selective column.

Q5.

Query B is better because it retrieves only the data the consumer needs. This can reduce I/O, network traffic, memory use, and query costs in some database systems.

It also makes the query more resilient to schema changes: adding an unrelated column to the table will not unexpectedly change the query's output.

Select * is convenient during exploration, but it may read or return columns the consumer does not need.

Q6.

```
SELECT
order_id,
customer_id,
order_date,
amount_gbp
FROM orders_perf
WHERE status = 'complete'
```

```
AND order_date >= '2026-08-03'
AND amount_gbp >= 40
ORDER BY order_date, order_id;
```

Q7.

```
CREATE TABLE customers_perf (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(100) NOT NULL,
city VARCHAR(50) NOT NULL
);
```

```
INSERT INTO customers_perf VALUES
(101, 'Amelia Clarke', 'London'),
(102, 'Oliver Bennett', 'Manchester'),
(103, 'Aisha Khan', 'Leeds'),
(104, 'Harry Thompson', 'Bristol'),
(105, 'Emily Wilson', 'Leeds'),
(106, 'Noah Davies', 'Bristol');
```

```
SELECT
o.order_id,
c.customer_name,
o.amount_gbp,
o.status
FROM orders_perf o
INNER JOIN customers_perf c
ON o.customer_id = c.customer_id;
```

Q8. Why should a query select only the columns it needs?

C. It can reduce unnecessary data read or transferred.

Q9.

order_date is the strongest starting choice because the reports commonly filter by date.

A date-based layout can allow systems to skip data from dates that are not needed.

The exact partition design depends on data volume and query patterns.

Q10.

```
orders/
├── order_date=2026-08-01/
├── order_date=2026-08-02/
└── order_date=2026-08-03/
```

Q11.

Design B is more likely to create excessive file-management and task overhead.

Why?

- Too many files: Design B contains 100,000 files, compared with just 100 files in Design A.
- File-management overhead: The system must track, list, open, and manage many more individual files.
- Task scheduling overhead: Distributed processing engines may create additional tasks to process numerous small files, increasing coordination costs.
- Slower processing: Time spent managing files and scheduling tasks can outweigh the time spent processing the actual data.

Q12. What is the main storage benefit of compression?

A. It reduces the amount of storage required for compressible data.

Q13.

Format	Text or binary?	Row or column oriented?	Common strength
CSV	Text	Row-oriented	Simple and widely supported
Parquet	Binary	Columnar	Analytical reads and compression
Avro	Binary	Row-oriented	Record exchange with schema
ORC	Binary	Columnar	Analytical storage and compression

Q14.

A. A simple file that a business user can open easily. —— CSV
B. Analytical data where queries often read only a few columns. —— Parquet
C. Record-oriented data exchange where schema travels with the data. —-Avro
D. Columnar analytical storage used with systems that support ORC well. —- ORC

Q15.

- CSV stores column names in the header, but it does not strongly store full data-type information.
- The file is human-readable.
- Values are stored as text.
- CSV does not provide the same columnar read benefits as Parquet or ORC.
- CSV is often larger than compressed binary formats for large analytical datasets.

Q16. Why is Parquet commonly used for analytical data? D. Its columnar layout can help queries read only needed columns.

Q17.

```
DROP INDEX idx_orders_customer
ON orders_perf;
```

```
EXPLAIN
SELECT
order_id,
order_date,
amount_gbp
FROM orders_perf
WHERE customer_id = 101
AND order_date >= '2026-08-02';
```

```
CREATE INDEX idx_orders_customer_date
ON orders_perf(customer_id, order_date);
```

```
EXPLAIN
SELECT
order_id,
order_date,
```

```
amount_gbp
FROM orders_perf
WHERE customer_id = 101
AND order_date >= '2026-08-02';
```

Q18.

Not necessarily.

The query needs to calculate a total across all order amounts and has no filter on `customer_id`.

The customer index does not directly reduce the number of rows needed for this result.

This is an example of why creating an index does not improve every query.

Q19.

Three improvements for BritMart's storage layout

1. Partition data by date: Organise orders into date-based partitions, such as by year and month. Since most reports filter by date, the system can skip irrelevant partitions and scan less data.

2. Reduce the number of tiny files: Compact the 500,000 small CSV files into fewer, larger files. This reduces file-management overhead and the number of processing tasks.

3. Use a columnar format such as Parquet or ORC: These formats are designed for analytical queries and allow the system to read only the required columns, reducing unnecessary data scanning.

Additional improvement: Apply compression where appropriate to reduce storage requirements and potentially lower data-transfer costs.

Q20.

Design choice	Recommendation	Reason
Partitioning approach	Partition by order_date, preferably by month if the data volume supports it	Reports filter by date, so partition pruning can reduce the amount of data scanned.
File format	Use Parquet or ORC	Columnar formats let analytical queries read only the required columns instead

Design choice	Recommendation	Reason
		of scanning all 60 columns.
Compression need	Enable suitable compression	Reduces storage usage over several years and can reduce data transfer. Balance compression costs against query performance.
Query habit	Select only the five required columns and filter by date	Avoid SELECT * and unnecessary data scans.

date-based partitioning + Parquet/ORC + compression + selective queries. Also compact small files and choose partition sizes based on actual data volume. Since BritMart retains several years of data, avoid overly fine partitions that create many tiny files.